---

**Taller: Protocolo Chord en Sistemas P2P**

*Arquitectura de Nube y Sistemas Distribuidos*
*Facultad de Ingeniería en TIC*
*Universidad Pontificia Bolivariana*
*Profesor: Álvaro Enrique Ospina*

---

## 1. Objetivo

En el presente taller nos proponemos:

- Explicar el problema de localización en redes P2P y cómo Chord lo resuelve.
- Construir y usar la *finger table* de un nodo.
- Determinar la ruta de enrutamiento (*lookup*) entre nodos.
- Describir el proceso de unión de nodos y actualización.
- Argumentar los *trade-offs* arquitectónicos y comunicar el análisis en un video breve.

---

## 2. Escenario Base

Se define:

- $m = 5$
- Espacio de identificadores: $0$a $2^5 - 1 = 31$
- Nodos activos: **1, 4, 8, 14, 21, 28**
- Regla: $successor(k) =$ primer nodo con ID $\geq k$ (con wrap-around)

---

## 3. Parte A — Finger Table (30%)

### A1. Construcción de la finger table para el nodo 8

La fórmula usada en Chord es:

$$start_i = (n + 2^{i-1}) \text{mod } 2^m$$
$$finger[i] = successor(start_i)$$

**Finger table para $n = 8$:**

| $i$ | $start_i$ | $successor(start_i)$ |
|---|---|---|
| 1 | 9 | 14 |
| 2 | 10 | 14 |
| 3 | 12 | 14 |
| 4 | 16 | 21 |
| 5 | 24 | 28 |

---

**A2. Interpretación de los intervalos**

Cada entrada cubre el intervalo: $[start_i, start_{i+1})$
Los intervalos van desde el nodo de referencia y avanzan en saltos de potencias de 2: $2^{(i-1)}$

Esto asegura:

- Cobertura primero en intervalos cercanos.
- Luego cobertura en intervalos lejanos.
- Crecimiento exponencial del alcance.

---

**¿Por qué la búsqueda es O(log N)?**

**Explicación verbal**

Suponiendo que partimos desde un nodo (p) hasta un nodo (q), existe una distancia (D). Usando *closest preceding finger*, por cada salto la distancia se reduce a la mitad o más. Por lo tanto, en el peor caso, la búsqueda es: complejidad $O(\log\ N)$

---

**Demostración matemática completa**

Cualquier número entero cumple que está entre dos potencias consecutivas de 2:

$$2^{(i-1)} \leq d < 2^i$$

donde: $d$= distancia entre $p$y $q$

Luego de un paso:

$$d_n = d - s$$

donde: $s$= salto realizado en la búsqueda.

Usando *closest preceding finger*, el salto es al menos:

$$s \geq 2^{(i-1)}$$

Como:

$$2^{(i-1)} \leq d$$

también sabemos que:

$$d < 2^i$$

Entonces:

$$\frac{d}{2} < 2^{(i-1)}$$

Sustituyendo:

$$d_n \leq d - 2^{(i-1)}$$
$$d_n < d - \frac{d}{2}$$
$$d_n < \frac{d}{2}$$

Es decir, en cada salto la nueva distancia es menor a la mitad de la anterior.

Aplicando esto a $t$ saltos:

$$d_t \leq \frac{d}{2^t}$$

Cuando:

$$\frac{d}{2^t} = 1$$
$$2^t = d$$
$$\log(2^t) = \log(d)$$
$$t = \log(d)$$

En el peor caso:

$$d \leq N$$

donde $N$es el total de nodos activos y además:

$$N \leq 2^m$$

Por tanto:

$$t = O(\log\ N)$$

---

**4. Parte B — Lookup (25%)**

⚠ Condición importante:
Para usar un valor de la finger table como *closest preceding finger*, debe pertenecer estrictamente al intervalo: $(n, K)$.

---

**B1. Búsqueda de K = 26 desde el nodo 8**

**Nodos activos**: 1 - 4 - 8 - 14 - 21 - 28

**Nodo inicial**: $n_1 = 8$

**Finger table para $n = 8$:**

| $i$ | $start_i$ | $successor(start_i)$ |
|---|---|---|
| 1 | 9 | 14 |
| 2 | 10 | 14 |
| 3 | 12 | 14 |
| 4 | 16 | 21 |
| 5 | 24 | 28 |

Intervalo válido: $(8, 26)$

El mayor nodo dentro del intervalo es: 21

Entonces: $8 \rightarrow 21$

---

**Finger table para $n = 21$:**

| $i$ | $start_i$ | $successor(start_i)$ |
|---|---|---|
| 1 | 22 | 28 |
| 2 | 23 | 28 |
| 3 | 25 | 28 |
| 4 | 29 | 1 |
| 5 | 5 | 8 |

No entra ninguno en $(21,26)$
Se pasa al sucesor inmediato: $21 \rightarrow 28$

Ahora: $K \in (predecesor(28), 28]$

El predecesor de 28 es 21.

$$successor(26) = 28$$

Ruta final: $8 \rightarrow 21 \rightarrow 28$

---

**B2. Búsqueda de K = 2 desde el nodo 4 (wrap-around)**

Nodos activos: 1 - 4 - 8 - 14 - 21 - 28

Sabemos: $successor(2) = 4$

Pero usando estrictamente *closest preceding finger*:

---

**Finger table para $n = 4$:**

| $i$ | $start_i$ | $successor(start_i)$ |
|---|---|---|
| 1 | 5 | 8 |
| 2 | 6 | 8 |
| 3 | 8 | 8 |
| 4 | 12 | 14 |
| 5 | 20 | 21 |

Todos son mayores que 2.

Sin embargo, el intervalo $(4,2)$ implica wrap-around: $(4 \rightarrow 31) \cup (0 \rightarrow 2)$

El nodo 21 pertenece a ese intervalo.

Entonces: $4 \rightarrow 21$

---

**Finger table para $n = 21$:**

| $i$ | $start_i$ | $successor(start_i)$ |
|---|---|---|
| 1 | 22 | 28 |
| 2 | 23 | 28 |
| 3 | 25 | 28 |
| 4 | 29 | 1 |
| 5 | 5 | 8 |

El nodo más cercano antes de 2 es 1.

Entonces: $21 \rightarrow 1$

---

**Finger table para $n = 1$:**

| $i$ | $start_i$ | $successor(start_i)$ |
|---|---|---|
| 1 | 2 | 4 |
| 2 | 3 | 4 |
| 3 | 5 | 8 |
| 4 | 9 | 14 |
| 5 | 17 | 21 |

Ninguno cumple $(1,2)$

Se pasa al sucesor inmediato: $1 \rightarrow 4$

Finalmente: $successor(2) = 4$

Ruta completa: $4 \rightarrow 21 \rightarrow 1 \rightarrow 4$

---

**5. Parte C — Join y Actualización (20%)**

Estado inicial:

1 - 4 - 8 - 14 - 21 - 28

Se agrega el nodo 18.

---

**C1. Proceso de unión**

El nodo 18 busca su sucesor como si fuera una llave normal usando *closest preceding finger*.

$$successor(18) = 21$$

Sabemos que:

$$predecessor(21) = 14$$

Entonces:

$$predecessor(18) = 14$$

21 actualiza:

$$predecessor(21) = 18$$

mediante $notify(18)$

Posteriormente:

$stabilize(14)$ consulta a 21 su predecesor.

Como es 18 y no 14:

$$successor(14) = 18$$

---

**Finger table para $n = 18$:**

| $i$ | $start_i$ | $successor(start_i)$ |
|---|---|---|
| 1 | 19 | 21 |
| 2 | 20 | 21 |
| 3 | 22 | 28 |
| 4 | 26 | 28 |
| 5 | 2 | 4 |

Inicialmente ningún finger externo apunta a 18.

Esto se corrige con $fix_fingers()$

---

**C2. Explicación de los algoritmos**

- $\boldsymbol{stabilize(n)}$: pregunta a $successor(n)$ su predecesor. Si es diferente, actualiza $successor(n)$
- $\boldsymbol{notify(n)}$: informa al sucesor para que actualice su predecesor si corresponde.
- $\boldsymbol{fix_fingers()}$: se ejecuta periódicamente y recalcula entradas de la finger table.

---

**6. Parte D — Análisis (15%)**

**¿Cuándo Chord es preferible a Cliente/Servidor?**

Es preferible cuando:

- Se requiere alta escalabilidad horizontal.
- No se desea un punto único de falla.
- Los nodos entran y salen dinámicamente.
- Se necesita distribución automática de carga.

En C/S:

- Existe cuello de botella.
- Escalar implica mayor costo.
- Hay dependencia central.

En Chord:

- Lookup es O(log N).
- Cada nodo enruta.
- Las claves se distribuyen automáticamente.

**¿Qué sacrifica?**

- Simplicidad arquitectónica.
- Consistencia fuerte inmediata.
- Latencia determinística.
- Control centralizado.

---

**Impacto del churn**

Alta rotación provoca:

- Finger tables desactualizadas.
- Rutas más largas.
- Reintentos.
- Mayor tráfico de mantenimiento.

- Mayor latencia real.
- Más consumo de red.
- Convergencia más lenta.

Chord funciona mejor con churn moderado.

---

**Despliegue en la nube**

La nube mitiga en C/S:

- Cuello de botella.
- Punto único de falla.
- Escalabilidad vertical limitada.

Pero permanecen:

- Centralización.
- Costos proporcionales.
- Coordinación entre réplicas.
- Dependencia del proveedor.

Chord:

- Distribuye carga orgánicamente.
- Es resiliente.
- Pero es más complejo de mantener.
- No garantiza consistencia fuerte inmediata.
- Es difícil de monitorear.

---

---

**Conclusión**

En el presente taller demostramos formalmente cómo Chord resuelve el problema de localización en sistemas P2P estructurados mediante hashing consistente y tablas de enrutamiento logarítmicas. Verificamos matemáticamente la complejidad ($O(\log N)$), analizamos rutas reales de búsqueda con y sin *wrap-around*, describimos el proceso de unión de nodos y evaluamos sus implicaciones arquitectónicas frente al modelo Cliente/Servidor.

Concluimos que Chord ofrece una solución altamente escalable y descentralizada, adecuada para entornos dinámicos, aunque introduce mayor complejidad operativa y estados transitorios de inconsistencia.

---

**Referencias**

Stoica, I., Morris, R., Karger, D., Kaashoek, M., & Balakrishnan, H. (2001). *Chord: A scalable peer-to-peer lookup protocol for Internet applications*. Proceedings of ACM SIGCOMM. Association for Computing Machinery. https://doi.org/10.1145/964723.383071

OpenAI. (2024). *ChatGPT (modelo GPT-5)* [Modelo de lenguaje de inteligencia artificial]. https://chatgpt.com/share/699e03be-ec6c-8002-9463-7a1f9c1708e2

---